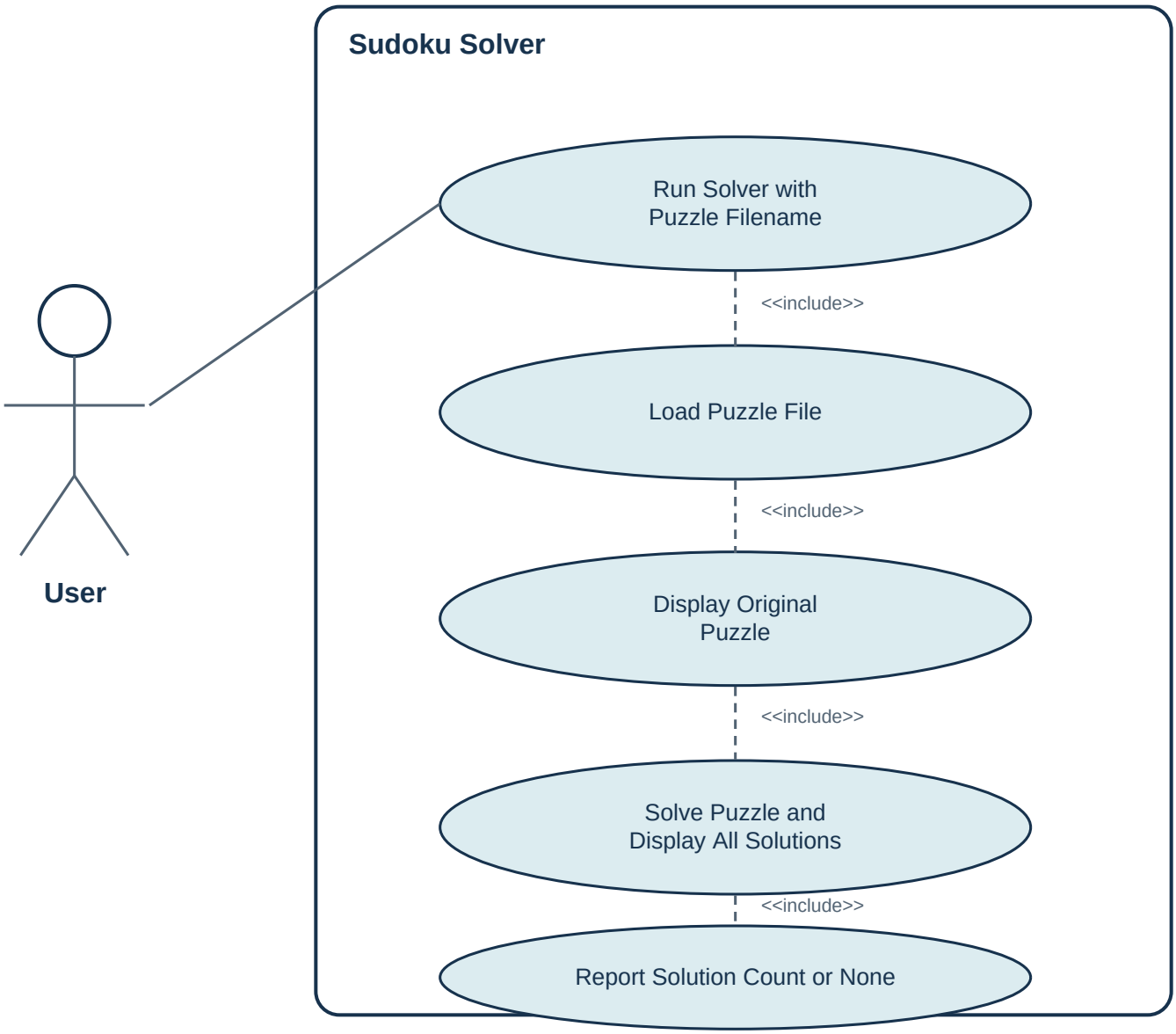
# EECS 348 | Assignment 4 - Sudoku Solver

## UML USE CASE DIAGRAM

System: Command-Line Sudoku Solver




### Primary actor: User

The user runs the program from the command line and provides the path to a Sudoku puzzle file. The program reads the puzzle, prints the original grid, checks its validity, and searches for all possible solutions.

Note: File loading, validation, and backtracking are internal steps of the main use case.

Lane Lamping | October 15, 2026

# USE CASE DETAILS

## 1. Identify a Use Case

Use Case: Solve a Sudoku Puzzle from a File

Primary Actor: User

System: Command-Line Sudoku Solver

## 2. Brief Description

The user provides a puzzle filename when starting the program. The program reads the 9x9 Sudoku grid, prints the original puzzle, checks for conflicts, and uses recursive backtracking to find and display every solution.

## 3. Step-by-Step Flow of Events

1. The user runs the executable with one puzzle filename as a command-line argument.
2. The program checks that exactly one filename was provided.
3. The program opens the file and reads the 81 puzzle cells.
4. The program prints the filename and original Sudoku grid.
5. The program checks whether the starting grid contains conflicting numbers.
6. If valid, the program finds an empty cell and tries digits 1 through 9.
7. For each valid digit, the program places it and calls solve() recursively.
8. When a completed board is found, the program prints it and increments the solution count.
9. The program backtracks to find any additional solutions.
10. The program prints the total solutions, or "No solution found" if there are none.

## 4. Detailed Description

Precondition: The program has been compiled and the user has a puzzle file containing a 9x9 grid. Digits 1-9 are filled cells and underscores represent blanks.

Main success scenario: The filename is accepted, the file is opened and read, and the starting puzzle has no conflicts. Recursive backtracking fills empty cells while checking the row, column, and 3x3 block. Every completed valid grid is printed. The final solution count is shown.

Alternate flow A - Incorrect arguments: If the user does not provide exactly one filename, main() prints the correct command format and exits.

Alternate flow B - File error: If the file cannot be opened or a required cell is missing or invalid, loadPuzzle() prints an error and run() stops.

Alternate flow C - No solution: If the starting board contains conflicts or the search finds no valid completed grid, the program prints "No solution found".

Postcondition: All solutions found for the selected puzzle have been displayed. The program then exits.